A Pipeline-Based Approach for Code-Switching Detection in Spanish-English Tweets

Andrew Geddes s5426375
Emiel Dost s4120140
Kylian de Rooij s4549503
Stijn Schreven s4804872

Abstract

Code-switching is the practice of alternating between languages within a conversation. It presents unique challenges for natural language processing (NLP). In this paper, we introduce a multi-stage pipeline for detecting code-switching in Spanish-English tweets. Our approach combines dictionary lookups, a fine-tuned BERT model and Named Entity Recognition (NER), with a key feature of our model being label locking to reduce error propagation. We evaluate our model on the Spanish-English LID dataset and demonstrate that our model significantly improves performance compared to the dictionary-based baseline. Our findings highlight the effectiveness of progressive label refinement in code-switching detection.

1 Introduction

Code-switching is the practice of alternating between two or more languages within a conversation. This is an increasingly common phenomenon in multilingual communities. Due to its informal nature, code-switching is a challenging task in natural language processing (NLP). This is due to frequent spelling and grammatical errors, lack of standardized spelling and excessive repetition of letters or punctiation. Accurate identification of code-switching is essential for processes like machine translation, conversational AI and sentiment analysis.

Code-switching makes traditional NLP tasks challenging, as most NLP models are strictly designed for monolingual tasks. In this paper, we present a multi-stage pipeline approach for code-switching in Spanish-English tweets. Our system progressively refines token classification through a series of steps. Using a dictionary-based lookup, a fine-tuned BERT model and named entity recognition (NER). A key feature of our model is label locking, which ensures that high-confidence labels are locked after each stage in the pipeline, reducing error propagation, and improving overall prediction stability.

Research Question In this paper, we investigate the following research question:

"How does a multi-stage pipeline with label locking improve code-switching detection in Spanish-English tweets?"

We will evaluate how each component in the pipeline contributes to the overall accuracy and whether progressive label refinement enhances classification performance compared to the baseline model.

Several approaches have been explored for code switching detection. Molina et al. (2016) provides an overview of various methods including statistical models, CRFs and deep-learning techniques. While their work displays the strengths and limitations of these approaches, we propose a different strategy: a multi-stage BERT-based pipeline with label locking. By progressively refining tokens, our approach aims to mitigate error propagation and improve prediction stability in Spanish-English tweets.

2 Data

We use the Spanish-English LID dataset (Molina et al., 2016), which is a widely used benchmark for code-switching detection. The labels in the dataset, along with their frequency, are shown in Table 1.

With the majority of the labels belonging to `lang1`, `lang2`, `ne` and `other`, as displayed in Table 1, these will be the main focus. The other labels contain much smaller numbers, suggesting that these categories are less frequent, but still relevant to the analysis.

The data shows a similar distribution of labels among the two data sets, allowing for effective model training and evaluation. Because the proportions are the same in both data sets, it allows for reliable comparisons and performance assessment.

Label	Train	Dev	Description
lang1	80,437	16,712	English words
lang2	112,988	14,955	Spanish words
ambiguous	266	39	Both languages
fw	23	2	Neither languages
mixed	42	6	Mixed languages
other	54,060	7,830	Non-words
unk	218	32	No language
ne	5,187	815	Named entities
total	253,221	40,391	

Table 1: Label Distribution in Training and Development Data

3 Method

In this study, we use a multi-stage pipeline to classify the tokens in a bilingual corpus. The methodology follows a series of preprocessing steps, dictionary lookups, a fine-tuned BERT classifier for further refining, and a Named Entity Recognition (NER) model. The key components of the method are outlined below.

3.1 Preprocessing

The preprocessing phase involves several steps to clean and prepare the data for classification. These steps happen during a single iteration over the data. We designed our model to skip over the redundant lines, so it is not necessary to remove these. Therefore we leave in the empty lines and lines containing `# sent_enum = ...`. Additionally, a token is automatically classified as 'other' when the token is a mention ('@'), emoji, or a URL. These 'other' labels are locked afterwards and cannot be overridden by the next steps in the pipeline.

3.2 Dictionary Lookup

To identify English ('lang1') and Spanish ('lang2') words, a dictionary lookup method is employed. We use the PyEnchant Python library for this task (Kelly and Contributors, 2025). Due to its simplicity, this strategy is only used to classify tokens when they appear in one of the dictionaries. For this task it is simple, yet effective.

Firstly, our model marks tokens with 2 or less characters with a placeholder, as they are too short to reliably predict. We use this placeholder when a token's label is yet to be determined. Likewise, words that appear in both dictionaries are also given the placeholder label. Following these steps, the words appearing in only one of the dictionaries are given their appropriate label. Finally, if the token does not meet any of these criteria, it is also given a placeholder label. When the dictionary lookup is complete, words with the labels `lang1`, `lang2` and `other` are locked for the following stages in the pipeline.

3.3 BERT Classifier

Once the dictionary lookup has been applied, a fine-tuned BERT model is used to predict the functional labels. For this task, we used the BERT multilingual base model (Devlin et al., 2018). This model was chosen because it is pre-trained on multiple languages, making it an excellent choice for handling code-switched data. Its ability to capture contextual information across different languages allows for more accurate token classification in mixed-language settings.

We fine-tuned our model to predict the labels `lang1`, `lang2`, `fw`, `mixed`, along with further refinement of the `unk` label. The BERT model predicts these labels based on the token's context within the sentence, which is what makes it a powerful model. A key requirement of our model is that it can only apply the label if it is above the confidence threshold of 0.5. This ensures that only confident labels are accepted, improving the overall accuracy of the classification.

3.4 Named Entity Recognition

The final stage of the pipeline is Named Entity Recognition (NER). NER is used to identify and classify proper nouns and named entities within the data. These entities can include names of people, locations, organizations or other specific terms. Unlike the BERT model, which focuses on contextual token classification, the NER model is specialized for detecting named entities, allowing it to capture cases that BERT might overlook.

The NER overwrites another label if its confidence is greater than the threshold of 0.65. This is the only stage in the pipeline that can overwrite locked labels. Similar to the BERT model, this guarantees a more confident prediction for improved classification accuracy.

3.5 Evaluation

The evaluation of the code-switching detection pipeline was based on a token-level comparison between our system's final output and a gold-standard reference. The annotated file dev.conll was used as the ground truth (gold standard), while the predictions produced by the pipeline were stored in

dev.final.conll.

We used the sklearn.metrics package to evaluate our scores, and Seaborn for the confusion matrix covering all label categories. This setup provides us a complete and clear basis for assessing the performance of the full pipeline.

4 Results

4.1 Baseline

The results of the baseline model are visualized in tables 2 and 3, displaying the evaluation metrics precision, recall, and F1-score per label. Additionally, a table containing the macro and micro average of the evaluation metrics is displayed. We observe relatively low macro and micro average scores for this model, indicating that it is not yet performing well.

The model achieved a high precision of 0.978 for the `lang1` label, which means it is generally accurate when it predicts `lang1`. However, recall is much lower at 0.593, indicating that many actual `lang1` tokens are missed. A similar pattern is observed for `lang2`, where precision is also high (0.977), but recall is even lower at 0.488. This suggests that the model is very conservative in labeling these two classes, leading to many false negatives.

Label	Precision	Recall	F1-score	Support
O	0.0000	0.0000	0.0000	0
ambiguous	0.0000	0.0000	0.0000	39
fw	0.0000	0.0000	0.0000	2
lang1	0.9775	0.5934	0.7385	16,712
lang2	0.9771	0.4877	0.6506	14,955
mixed	0.0000	0.0000	0.0000	6
ne	0.0000	0.0000	0.0000	815
other	0.0000	0.0000	0.0000	7,830
unk	0.0000	0.0000	0.0000	32

Table 2: Evaluation metrics per label on the development set (baseline model)

Several labels received zero scores across all evaluation metrics: `ambiguous`, `fw`, `mixed`, `ne`, `other`, and `unk`. This is because this baseline model lacks specific mechanisms to identify them. For example, other and ne are often classified as `O` (which is a placeholder for unknown tokens) because the model does not include named entity recognition or a filter for non-language tokens.

The macro average recall of just 0.120 highlights the model's inability to generalize across all label types, while the micro average F1-score of 0.426 reflects its high scores predicting the `lang1` and `lang2` labels. These results underline the limitations of this baseline and provides us a base from which we can add to the pipeline.

Metric	Precision	Recall	F1-score
Micro Average	0.4261	0.4261	0.4261
Macro Average	0.2172	0.1201	0.1543

Table 3: Global evaluation metrics of the baseline model on the development set

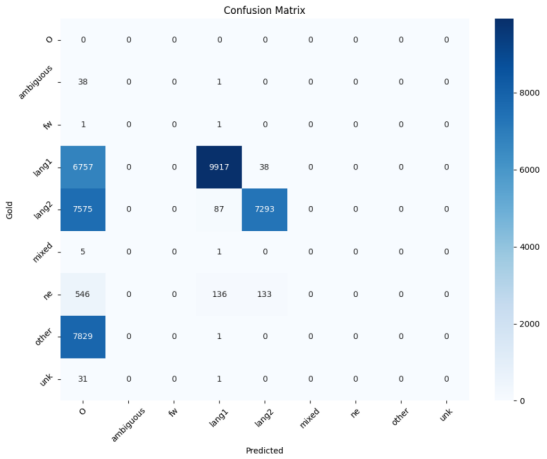

Figure 1: Confusion matrix of baseline model predictions on the development set

4.2 BERT-based model

The results of the BERT-based pipeline model are shown below using the same structure as for the baseline: tables for precision, recall, and F1-score per label, an overview of the global metrics, and a confusion matrix. Compared to the baseline, the model performs significantly better across nearly all metrics. Both the macro and micro averages are much higher, with a micro F1-score of 0.9674 indicating strong overall performance.

Label	Precision	Recall	F1-score	Support
ambiguous	0.0000	0.0000	0.0000	39
fw	0.0000	0.0000	0.0000	2
lang1	0.9768	0.9695	0.9732	16,712
lang2	0.9779	0.9712	0.9745	14,955
mixed	0.0000	0.0000	0.0000	6
ne	0.8211	0.6699	0.7378	815
other	0.9540	0.9960	0.9746	7,830
unk	0.0180	0.0625	0.0280	32

Table 4: Evaluation metrics per label on the development set (BERT-based model)

We observe an improvement in the prediction of `lang1` and `lang2`, which are also the most frequent

classes in the dataset. `lang1` reaches a precision of 0.977 and recall of 0.970, meaning the model is both accurate and consistent in identifying tokens of this class. `lang2` performs similarly, with a precision of 0.978 and a recall of 0.971. This balance between precision and recall shows that the model has learned to distinguish these two language classes very effectively.

Metric	Precision	Recall	F1-score
Micro Average	0.9674	0.9674	0.9674
Macro Average	0.4685	0.4587	0.4610

Table 5: Global evaluation metrics of the BERT-based model on the development set

The confusion matrix also confirms that many of the issues present in the baseline have been addressed. There are very few misclassifications between `lang1` and `lang2`, and the label `unk` is rarely predicted. This may suggest that the BERT-based model is better at handling informal or misspelled tokens which tend to be frequent on twitter, where the dictionary lookup in the baseline failed.

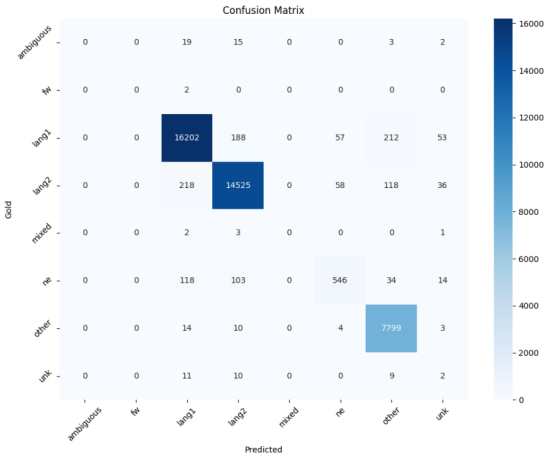

Figure 2: Confusion matrix of final BERT-based predictions on the development set

Named Entity Recognition also contributes positively in this pipeline. The `ne` label achieves a precision of 0.821 and recall of 0.670, which is a significant improvement considering that this label was not recognized at all in the baseline. This improvement results from the NER-specific override step, which allows the model to revise earlier label predictions when sufficient confidence is achieved.

Some of the smaller categories, such as `fw`, `mixed`, `ambiguous`, and `unk`, remain challenging. This is not surprising, as these labels have very low support in the dataset, making it difficult for the model to learn robust patterns without additional data or targeted features.

Overall, the BERT-based pipeline benefits from several important design choices: progressive label refinement, label locking, and the integration of contextual understanding through the BERT model. Each of these elements contributes to improved prediction accuracy and helps reduce the kinds of errors observed in the baseline model.

5 Conclusion

In this paper, we explored a multi-stage pipeline approach for code-switching detection in Spanish-English tweets. Our results show that progressively refining token classification through dictionary-based lookup, a fine-tuned BERT model and named entity recognition significantly improves accuracy. Additionally, label locking proved to be a very efficient mechanism to stabilize label prediction and reduce misclassifications.

To answer our research question:

"How does a multi-stage pipeline with label locking improve code-switching detection in Spanish-English tweets?"

Our findings suggest that this approach improves classification accuracy by incrementally refining predictions while reducing error propagation. The label locking mechanism ensures that high confidence labels remain stable throughout the process making the final classifications more reliable

This study highlights the potential of a multi-stage pipeline for code-switching detection. By showing consistent accuracy gains through progressive refinement, we provide a strong foundation for further research into more sophisticated pipeline approaches.

Limitations

Despite the strong performance of the BERT-based model, several limitations still remain. The gold standard contains inconsistencies in annotations, potentially affecting model performance and evaluation reliability. Additionally, the dataset is very limited for rare categories, causing difficulties for the model to learn any meaningful patterns for these labels. Addressing these issues could further improve the model's accuracy and performance.

Individual Contributions

Throughout the project, our team had regular meetings to discuss our progress, address challenges, and plan for our next steps. Each member actively contributed by providing insights and insightful feedback, ensuring continuous improvement in our approach. Kylian initiated the preprocessing stage, preparing the data for further processing. Andrew and Emiel focused on developing the pipeline model, implementing and optimizing each stage of the classification. Stijn and Kylian collaborated on writing the report, structuring our findings and documenting the method, results and analysis. This collaborative effort ensured a well-rounded and thouroughly evaluated approach to code-switching detection.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805.

Ryan Kelly and Contributors. 2025. Pyenchant: Spellchecking library for python. https://pypi.org/project/pyenchant/. Version X.Y.Z, accessed March 29, 2025.

Giovanni Molina, Fahad AlGhamdi, Mahmoud Ghoneim, Abdelati Hawwari, Nicolas Rey-Villamizar, Mona Diab, and Thamar Solorio. 2016. Overview for the second shared task on language identification in code-switched data. In *Proceedings of the Second Workshop on Computational Approaches to Code Switching*, pages 40–49, Austin, Texas. Association for Computational Linguistics.